

13. NodeJS – syntaxe, cesty, zpracování požadavků

Co je NodeJS?

- Běhové prostředí pro JS
- Umožňuje spustit JS kód mimo prohlížeč
- Postavený nad JS enginem V8
- Event-driven, I/O non-blocking
- Async zpracování požadavků
- Zvládá hodně požadavků najednou, velká škálovatelnost

Moduly a NPM packages

- Základní bloky aplikací
- Organizují kód do logických komponent
- Každý soubor = modul
- Je to jako import v jiných jazycích
- Dvě možnosti zápisu:
 - CommonJS
 - Starší, výchozí nastavení
 - Zápis: `const fs = require('fs');`
 - ES Modules
 - Moderní
 - Zápis: `import fs from 'fs';`
- Pokud je import z 3rd party zdroje tak nutno instalovat přes npm v konzoli
- Má i build-in moduly, které nejsou nutné instalovat (fs, http, path, url, event, os)

NPM packages

- NPM = package manager pro NodeJS (stahuje package)
- Package = soubor modulů (souborů)
- Použití v konzoli: `npm install upper-case`
- Uloží se do složky `node_modules`

package.json

- Soubor, popisující projekt
- Obsahuje název, verzi, popis app, dependencies
- Když nainstalujeme package tak se automaticky připsá do dependencies

Vytvoření serveru

HTTP Modul

- Pro tvorbu HTTP serveru
- Přijímá requesty a odesílá odpovědi
- Umí HTTP požadavky na jiné servery
- GET, POST, PUT, DELETE, ... metody
- Hodně ručně dělaných věcí

```
// Import the HTTP module
const http = require('http');

// Create a server object
const server = http.createServer((req, res) => {
  // Set the response HTTP header with HTTP status and Content type
  res.writeHead(200, { 'Content-Type': 'text/plain' });

  // Send the response body as 'Hello, World!'
  res.end('Hello, World!\n');
});

// Define the port to listen on const PORT = 3000;

// Start the server and listen on the specified port
server.listen(PORT, 'localhost', () => {
  console.log(`Server running at http://localhost:${PORT}/`);
});
```

ExpressJS

- Framework
- Lehký, rychlý
- Standardizovaný kód, modulární architektura
- Hotové nástroje pro zpracování požadavků
- Lepší pro reálné využití

```
const express = require('express');
const app = express();
const port = 8080;

// Define a route for GET requests to the root URL
app.get('/', (req, res) => {
  res.send('Hello World from Express!');
});

// Start the server
app.listen(port, () => {
  console.log(`Example app listening at http://localhost:${port}`);
});
```

Cesty

- Mapování konkrétní URL adresy a HTTP metody na konkrétní blok kódu
- V Expressu se definují přímo přes objekt aplikace (např. `app.get()`)
- Zápis: `app.get('/cesta', (req, res) => { /* kód */ })`
- Express router - umožňuje oddělit cesty do samostatných souborů pomocí router objektu (`express.Router()`)

Zpracování požadavků

- *req.params*: Slouží ke čtení dynamických částí URL adresy (tzv. route parametry). Z cesty `/users/:id` přečteme ID jako `req.params.id`
- *req.query*: Slouží ke čtení parametrů za otazníkem (tzv. query string). Z URL `/users?sort=asc` přečteme hodnotu jako `req.query.sort`
- *req.body*: Slouží ke čtení dat zaslaných v těle požadavku (typicky při metodách POST nebo PUT)

Odpovědi

- *res.send()*: Základní metoda pro odeslání odpovědi (pošle prostý text nebo HTML).
- *res.json()*: Metoda, která JavaScriptový objekt automaticky převede na JSON a správně nastaví hlavičku Content-Type
- *res.status()*: Nastavení HTTP stavového kódu odpovědi. Lze řetězit (např. `res.status(404).json({ chyba: 'Nenalezeno' })`)

Middleware

- Funkce, která zachytí požadavek "napůl cesty" – spustí se mezi přijetím požadavku a odesláním konečné odpovědi
- Přijímá 3 parametry: (*req*, *res*, *next*)
- *next()*: Klíčová metoda. Její zavolání v middlewaru volá v Expressu další požadavek nebo cíovou cestu
- Globálně nebo jen pro nějakou cestu
- Ověření přihlášení, logování, parsování JSONu

Asynchronní zpracování

- Dotazy do databáze nebo čtení souborů nesmí blokovat Event Loop
- `async/await`
- zapisuje se před async operaci
- vždy nutné obalit do try catch bloku pro chytání chyb